Table des matières

Labo 07 — Traiter un gros volume 1
Avant de commencer 2
Étape 1 — Demander tout l'historien, et refaire le bronze sur une machine 3
Étape 2 — Les agrégats sur une machine, et le plan qui les explique 4
Étape 3 — Le même bronze, en PySpark, dans un conteneur à usage unique 5
Étape 4 — Comparer ce que les deux moteurs ont écrit 8
Étape 5 — Les agrégats en Spark, le brassage dans le plan, et la comparaison ligne à ligne 8
Étape 6 — Dédoublonner la télémétrie, le pire cas pour Spark 11
Étape 7 — Les trois pièges, mesurés 13
Étape 8 — Conclure, remettre le projet d'aplomb, versionner 15
Dépannage 17
Annexe — Sur Databricks 18

Labo 07 — Traiter un gros volume

Labo · Cours 7/12 — Formation Data Engineering

But : demander à l'usine tout son historien et refaire le bronze avec le pipeline ; mesurer le travail sur une machine ; écrire le même en PySpark et le lancer dans un conteneur à usage unique ; comparer les résultats, les durées, les fichiers et les plans ; provoquer les trois pièges ; remettre le projet dans l'état du Cours 06 et versionner. Le pourquoi de chaque étape est dans `Theorie_07_DE.md`, section 9 et annexe A.

Le labo se fait en deux séances, coupées au point d'arrêt de l'étape 5 : la première compare les deux moteurs, la seconde ajoute le pire cas, les pièges et la remise en état.

Trois fenêtres. Chacune reste dans `Projet_Boreal_intermediaire` du début à la fin du labo : aucun `cd` à taper après l'ouverture.

Fenêtre	Dossier	Ce qu'on y fait
1. Anaconda Prompt, `formations` actif	`Projet_Boreal_intermediaire`	le Bloc-notes, le générateur, le pipeline, les mesures de DuckDB (`duckdb -c`), Git
2. Anaconda Prompt devenue Ubuntu	`Projet_Boreal_intermediaire`	Docker, donc Spark

© 2026 iAlexMG. Tous droits réservés.

Fenêtre	Dossier	Ce qu'on y fait
3. Anaconda Prompt, à l'étape 2	`Projet_Boreal_intermediaire`	l'interface web de DuckDB (`duckdb -ui`), pour les contrôles

Liste de contrôle :

- ☐ Image de Spark tirée : `spark:4.2.0-scala2.13-java21-python3-ubuntu`, 2,22 Go
- ☐ Sources régénérées avec l'historien complet : 2 628 fichiers, empreinte `0e53ccf4c7b8e73b…`
- ☐ Bronze refait sur une machine : 2 612 281 lignes, source et bronze d'accord
- ☐ Agrégats sur une machine : 47 618 lignes, 2 405 366 points ; `PARQUET_SCAN` et son filtre poussé
- ☐ Bronze écrit par Spark : 27 partitions de lecture, 2 407 796 lignes
- ☐ Deux bronzes comparés : mêmes lignes, 12,7 Mo en zstd contre 18,8 Mo en snappy
- ☐ Agrégats en Spark : les mêmes comptes, 6 moyennes à un millième, un `Exchange` dans le plan
- ☐ Télémétrie dédoublonnée des deux côtés : 201 542 messages
- ☐ Les trois pièges mesurés
- ☐ Projet revenu à l'état du Cours 06 : 1 989 fichiers, historien de 550 087 lignes
- ☐ `spark/` et `.gitattributes` versionnés dans `01_de`, `git status` propre

Avant de commencer

Fenêtre 1, une Anaconda Prompt, dans `Projet_Boreal_intermediaire` :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
set PYTHONPATH=01_de
REM Attendu : rien. Python cherchera aussi ses paquets dans 01_de.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
```

Fenêtre 2 : une nouvelle Anaconda Prompt, dans `Projet_Boreal_intermediaire`, puis :

© 2026 iAlexMG. Tous droits réservés.

```
wsl
```

Attendu : une invite qui finit par …/Projet_Boreal_intermediaire$. Cette fenêtre est désormais Ubuntu, dans le projet, et y reste jusqu'à la fin du labo.

Vérifier que rien ne tourne, puis tirer l'image de Spark. Fenêtre 2 :

```
docker ps
# Attendu : l'en-tête seul. La pile d'Airflow du Cours 06 est arrêtée, et elle le reste.
docker pull spark:4.2.0-scala2.13-java21-python3-ubuntu
# Attendu : Status: Downloaded newer image la première fois, en une minute et demie environ ;
# Status: Image is up to date ensuite.
docker images spark
# Attendu : spark:4.2.0-scala2.13-java21-python3-ubuntu, 2.22GB sous DISK USAGE.
```

Étape 1 — Demander tout l'historien, et refaire le bronze sur une machine

Théorie : Theorie_07_DE.md, annexe A.1.

1.1 Régénérer les sources avec l'historien complet. Jusqu'ici, l'usine livrait 182 jours d'historien ; `--historien complet` ouvre ses archives depuis la mise en service du SCADA, le 2024-01-01, et `--jusqua 2026-03` garde les mois de 2026 du Cours 06. Fenêtre 1 :

```
python ..\_univers\boreal\generer_boreal.py --projet . --jusqua 2026-03 --historien complet
REM Attendu, en deux minutes et demie environ : le bilan, dont historien 821 jours, 2 407 796
REM lignes, puis 2 628 fichiers, 269,9 Mo, empreinte globale 0e53ccf4c7b8e73b..., et Conforme à
REM la référence des cours.
```

1.2 Refaire le bronze avec le pipeline du Cours 05 : les Parquet d'hier ne se sont pas mis à jour tout seuls. Fenêtre 1 :

```
python -m pipeline bronze
REM Attendu : historien 2 407 796 lignes et télémétrie 204 485 lignes, source et bronze
REM d'accord, puis étape bronze (...) : fin, 2 612 281 lignes.
```

1.3 Chronométrer la conversion seule, par le script du Cours 02, que le pipeline lance aussi. Fenêtre 1 :

```
duckdb -cmd ".timer on" -f 01_de\sql\bronze_historien.sql
REM Attendu : Run Time (s): real 2.7 environ pour le COPY, puis 2407796 des deux côtés du
REM contrôle, qui prend une vingtaine de secondes.
```

`.timer on` fait suivre chaque requête du script de sa durée. Le commentaire du script annonce 275 436 lignes : c'était la génération par défaut du Cours 02.

© 2026 iAlexMG. Tous droits réservés.

La commande	Ce qu'elle fait	Chez l'auteur
`python -m pipeline bronze` (1.2)	les deux bronzes, chacun avec son contrôle, et le journal des exécutions	29,8 s
`duckdb -f 01_de\sql\bronze_historien.sql` (1.3)	le `COPY` de l'historien seul, puis son contrôle	2,7 s pour le `COPY`

Retenir ce 2,7 : c'est la barre que Spark devra franchir.

Étape 2 — Les agrégats sur une machine, et le plan qui les explique

Théorie : `Theorie_07_DE.md`, annexe A.2.

2.1 Créer le dossier de comparaison, hors du dépôt. Fenêtre 1 :

```
mkdir comparaison\duckdb
REM Attendu : rien.
```

2.2 Calculer les agrégats quotidiens par étiquette, chronomètre en marche : par étiquette et par jour, le nombre de points de qualité bonne, leur moyenne, leur minimum et leur maximum. Fenêtre 1 :

```
duckdb -cmd ".timer on" -c "COPY (SELECT TagName AS etiquette, CAST(Timestamp AS DATE) AS jour, COUNT(*) AS points,
ROUND(AVG(Value), 3) AS moyenne, MIN(Value) AS minimum, MAX(Value) AS maximum FROM
'plateforme/lac/bronze/usine/historien/*/*.parquet' WHERE Quality = 192 GROUP BY TagName, CAST(Timestamp AS DATE)) TO
'comparaison/duckdb/agregats.parquet' (FORMAT parquet);"
REM Attendu : Run Time (s): real 0.19 environ.
```

2.3 Ouvrir l'interface web de DuckDB. Fenêtre 3 : une nouvelle Anaconda Prompt, dans `Projet_Boreal_intermediaire`, puis :

```
conda activate formations
duckdb -ui
```

Laisser cette fenêtre ouverte : la fermer arrête l'interface. Créer un notebook neuf (+ à côté de Notebooks) et le renommer « Labo 07 » (double-clic sur son titre, puis Entrée). Une requête par cellule, Ctrl+Entrée pour l'exécuter.

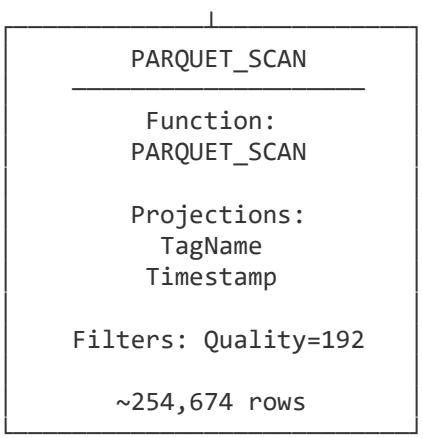
2.4 Contrôler ce que le moteur d'une machine a produit.

```
-- Attendu : 47618 agrégats, 2405366 points, somme des moyennes 2737589.987.
SELECT COUNT(*) AS agregats, SUM(points) AS points, ROUND(SUM(moyenne), 3) AS somme_moyennes
FROM 'comparaison/duckdb/agregats.parquet';
```

2.5 Demander le plan du même regroupement, sans l'exécuter. Fenêtre 1 :

© 2026 iAlexMG. Tous droits réservés.

```
duckdb -c "EXPLAIN SELECT TagName, CAST(Timestamp AS DATE) AS jour, COUNT(*) AS points FROM
'plateforme/lac/bronze/usine/historien/*/*.parquet' WHERE Quality = 192 GROUP BY TagName, CAST(Timestamp AS DATE);"
REM Attendu : un plan à lire de bas en haut, HASH_GROUP_BY, PROJECTION, puis ce dernier bloc.
```



Deux colonnes remontées, le filtre appliqué dans le fichier, une estimation tirée des statistiques du Parquet ; au-dessus, un `HASH_GROUP_BY`, et rien qui se déplace.

Étape 3 — Le même bronze, en PySpark, dans un conteneur à usage unique

Théorie : `Theorie_07_DE.md`, annexe A.3.

3.1 Créer le dossier du code de Spark, dans le dépôt, à côté du paquet `pipeline`. Fenêtre 1 :

```
mkdir 01_de\spark
REM Attendu : rien.
```

3.2 Écrire le lanceur, qui décide de tout ce qui entoure Spark. Fenêtre 1 :

```
notepad 01_de\spark\spark.sh
```

Accepter la création. Coller, enregistrer, fermer :

```
#!/bin/bash
# spark.sh : lance un script PySpark dans un conteneur à usage unique (Cours 07).
#
# Ce que fait chaque option :
#   --rm          le conteneur disparaît à la fin ; rien ne tourne entre deux lancements
#   --user root   Spark change les droits des dossiers qu'il crée, ce qu'un disque Windows monté
```


© 2026 iAlexMG. Tous droits réservés.

```
#                 refuse à tout autre utilisateur
#   -v "$PWD"     le projet entre dans le conteneur, à l'endroit que -w rend courant
#   local[*]      un pilote et des exécuteurs dans le même processus, sur tous les cœurs
set -e
docker run --rm \
  --user root \
  -v "$PWD":/projet \
  -w /projet \
  spark:4.2.0-scala2.13-java21-python3-ubuntu \
  /opt/spark/bin/spark-submit --master "local[*]" "$1"
```

3.3 Écrire la conversion de l'historien, le même travail qu'en 1.3. Fenêtre 1 :

```
notepad 01_de\spark\convertir_historien.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""convertir_historien.py : le bronze de l'historien, écrit par Spark (Cours 07)."""
import time

from pyspark.sql import SparkSession, functions as F, types as T

SOURCE = "sources/usine/historien/*.csv"
SORTIE = "comparaison/spark/historien"

# Le schéma se déclare, comme au Cours 02 : deviner coûte une lecture de plus et se trompe.
SCHEMA = T.StructType([
    T.StructField("TagName", T.StringType()),
    T.StructField("Timestamp", T.TimestampType()),
    T.StructField("Value", T.DoubleType()),
    T.StructField("Quality", T.IntegerType()),
])

spark = SparkSession.builder.appName("boreal-bronze-historien").getOrCreate()
spark.sparkContext.setLogLevel("WARN")

historien = (spark.read
             .option("header", True)
             .option("timestampFormat", "yyyy-MM-dd HH:mm:ss")
             .schema(SCHEMA)
             .csv(SOURCE)
             # historien_2025-10-28.csv donne la partition jour=2025-10-28
```


© 2026 iAlexMG. Tous droits réservés.

```
            .withColumn("jour", F.to_date(F.regexp_extract(
                F.input_file_name(), r"historien_(\d{4}-\d{2}-\d{2})", 1))))

print("cours 07 | partitions de lecture :", historien.rdd.getNumPartitions())
print("cours 07 | partitions de brassage :", spark.conf.get("spark.sql.shuffle.partitions"))
print("cours 07 | cœurs vus :", spark.sparkContext.defaultParallelism)

debut = time.perf_counter()
historien.write.mode("overwrite").partitionBy("jour").parquet(SORTIE)
print(f"cours 07 | conversion : {time.perf_counter() - debut:.1f} s")

# Contrôle : autant de lignes écrites que reçues.
# Attendu : 2 407 796, le compte du bilan de la génération.
debut = time.perf_counter()
lignes = spark.read.parquet(SORTIE).count()
print(f"cours 07 | lignes écrites : {lignes} (relues en {time.perf_counter() - debut:.1f} s)")

spark.stop()
```

3.4 Erreur voulue : lancer la conversion avec le lanceur tel que le Bloc-notes l'a enregistré. Fenêtre 2 :

```
bash 01_de/spark/spark.sh 01_de/spark/convertir_historien.py
# Attendu : une erreur. La ligne 10 annonce set: - et invalid option, brouillée à l'écran ;
# puis docker: invalid reference format ; puis les lignes 12 à 16 en command not found.
# Spark ne démarre pas.
```

À retenir — le Bloc-notes termine chaque ligne par un retour chariot suivi d'un saut de ligne, à la mode de Windows, et bash prend ce retour chariot pour un caractère de la commande : il lit `set -e` suivi d'un caractère invisible. Le message ne nomme jamais la cause. On provoque l'erreur ici parce que tout script shell écrit sous Windows et lancé sous Linux la rencontre, au poste comme en entreprise.

3.5 Retirer les retours chariot du lanceur. Fenêtre 2 :

```
sed -i 's/\r$//' 01_de/spark/spark.sh
# Attendu : rien.
```

3.6 Relancer la conversion, pour prouver que le retour chariot était la seule cause. Fenêtre 2 :

```
bash 01_de/spark/spark.sh 01_de/spark/convertir_historien.py
# Attendu, en une minute et demie à deux minutes : partitions de lecture : 27, partitions de
# brassage : 200, cœurs vus : le nombre de cœurs du poste, conversion : entre 45 et 80 s,
# lignes écrites : 2407796.
```

Spark écrit son propre journal dans la fenêtre, quelques centaines de lignes en anglais : les lignes du script commencent toutes par `cours 07 |`, pour qu'on les retrouve. Dans le journal, un `FileNotFoundException` sur `sources/usine/historien/*.csv` et des `WARN MemoryManager` sont sans conséquence.

© 2026 iAlexMG. Tous droits réservés.

3.7 Relancer une fois de plus, pour voir la durée changer d'un passage à l'autre. Fenêtre 2 :

```
bash 01_de/spark/spark.sh 01_de/spark/convertir_historien.py
# Attendu : les mêmes 2407796 lignes, et une autre durée de conversion.
```

Étape 4 — Comparer ce que les deux moteurs ont écrit

Théorie : `Theorie_07_DE.md`, annexe A.4.

4.1 Lister une journée du bronze écrit sur une machine. Fenêtre 1 :

```
dir "plateforme\lac\bronze\usine\historien\jour=2025-10-28"
REM Attendu : un fichier, data_0.parquet, 22 277 octets.
```

4.2 Lister la même journée, écrite par Spark. Fenêtre 1 :

```
dir "comparaison\spark\historien\jour=2025-10-28"
REM Attendu : part-00000-<identifiant>.c000.snappy.parquet, 31 756 octets, et le même nom
REM précédé d'un point et suivi de .crc, 260 octets.
```

Les guillemets comptent : sans eux, l'Anaconda Prompt coupe le chemin au signe =, et répond « Fichier introuvable ».

4.3 Peser les deux bronzes. Fenêtre 2 :

```
du -sh --apparent-size plateforme/lac/bronze/usine/historien comparaison/spark/historien
# Attendu : 13M pour le premier, 19M pour le second.
```

`--apparent-size` additionne la taille des fichiers, et non la place qu'ils occupent sur le disque.

4.4 Compter les lignes des deux bronzes, chacun de son côté. Fenêtre 3 :

```
-- Attendu : 2407796 et 2407796. Un écart : un des deux n'a pas été refait.
SELECT
    (SELECT COUNT(*) FROM 'plateforme/lac/bronze/usine/historien/*/*.parquet') AS une_machine,
    (SELECT COUNT(*) FROM 'comparaison/spark/historien/*/*.parquet')           AS spark;
```

Étape 5 — Les agrégats en Spark, le brassage dans le plan, et la comparaison ligne à ligne

Théorie : `Theorie_07_DE.md`, annexe A.5.

5.1 Écrire les agrégats en PySpark. Fenêtre 1 :

```
notepad 01_de\spark\agregats_usine.py
```


© 2026 iAlexMG. Tous droits réservés.

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""agregats_usine.py : les agrégats quotidiens par étiquette, en Spark (Cours 07)."""
import time

from pyspark.sql import SparkSession, functions as F

BRONZE = "comparaison/spark/historien"
SORTIE = "comparaison/spark/agregats"

spark = SparkSession.builder.appName("boreal-agregats-usine").getOrCreate()
spark.sparkContext.setLogLevel("WARN")

agregats = (spark.read.parquet(BRONZE)
            .filter(F.col("Quality") == 192)
            .groupBy("TagName", F.to_date("Timestamp").alias("jour"))
            .agg(F.count("*").alias("points"),
                 F.round(F.avg("Value"), 3).alias("moyenne"),
                 F.min("Value").alias("minimum"),
                 F.max("Value").alias("maximum")))

print("cours 07 | le plan, avant de lancer quoi que ce soit :")
agregats.explain(mode="simple")

debut = time.perf_counter()
agregats.write.mode("overwrite").parquet(SORTIE)
print(f"cours 07 | agrégats : {time.perf_counter() - debut:.1f} s")

# Contrôle : les mêmes comptes que sur une machine.
# Attendu : 47 618 agrégats et 2 405 366 points. Un écart : le bronze n'a pas été refait.
controle = (spark.read.parquet(SORTIE)
            .agg(F.count("*").alias("agregats"),
                 F.sum("points").alias("points"),
                 F.round(F.sum("moyenne"), 3).alias("somme_moyennes"))
            .collect()[0])
print(f"cours 07 | agrégats : {controle['agregats']}, points : {controle['points']}, "
      f"somme des moyennes : {controle['somme_moyennes']}")

spark.stop()
```

5.2 Lancer les agrégats. Le plan sort avant que rien ne soit calculé. Fenêtre 2 :

© 2026 iAlexMG. Tous droits réservés.

```
bash 01_de/spark/spark.sh 01_de/spark/agregats_usine.py
# Attendu : le plan ci-dessous, puis agrégats : une dizaine de secondes, puis agrégats : 47618,
# points : 2405366, somme des moyennes : 2737589.989.
```

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[TagName#0, _groupingexpression#20], functions=[count(1), avg(Value#2), min(Value#2), max(Value#2)])
   +- Exchange hashpartitioning(TagName#0, _groupingexpression#20, 200), ENSURE_REQUIREMENTS, [plan_id=15]
      +- HashAggregate(keys=[TagName#0, _groupingexpression#20], functions=[partial_count(1), partial_avg(Value#2),
partial_min(Value#2), partial_max(Value#2)])
         +- Project [TagName#0, Value#2, cast(Timestamp#1 as date) AS _groupingexpression#20]
            +- Filter (isnotnull(Quality#3) AND (Quality#3 = 192))
               +- FileScan parquet [TagName#0,Timestamp#1,Value#2,Quality#3,jour#4] Batched: true, DataFilters:
[isnotnull(Quality#3), (Quality#3 = 192)], Format: Parquet, Location: InMemoryFileIndex(1
paths)[file:/projet/comparaison/spark/historien], PartitionFilters: [], PushedFilters: [IsNotNull(Quality),
EqualTo(Quality,192)], ReadSchema: struct<TagName:string,Timestamp:timestamp,Value:double,Quality:int>
```

Les numéros qui suivent # sont les identifiants internes des colonnes, et `_groupingexpression` est la colonne du jour, calculée par le `Project`. La ligne qui n'existe pas sur une machine est l'`Exchange` : le brassage, encadré de deux `HashAggregate`.

5.3 Comparer les deux jeux d'agrégats, ligne à ligne. Fenêtre 3 :

```
-- Attendu : 47618 paires, 0 écart sur les points et les extrêmes, 6 moyennes différentes.
WITH une_machine AS (SELECT etiquette, jour, points, moyenne, minimum, maximum
                     FROM 'comparaison/duckdb/agregats.parquet'),
     spark AS (SELECT TagName AS etiquette, jour, points, moyenne, minimum, maximum
               FROM 'comparaison/spark/agregats/*.parquet')
SELECT
    (SELECT COUNT(*) FROM une_machine JOIN spark USING (etiquette, jour)) AS paires,
    (SELECT COUNT(*) FROM une_machine JOIN spark USING (etiquette, jour)
      WHERE une_machine.points <> spark.points) AS points_differents,
    (SELECT COUNT(*) FROM une_machine JOIN spark USING (etiquette, jour)
      WHERE une_machine.moyenne IS DISTINCT FROM spark.moyenne) AS moyennes_differentes,
    (SELECT COUNT(*) FROM une_machine JOIN spark USING (etiquette, jour)
      WHERE une_machine.minimum IS DISTINCT FROM spark.minimum
         OR une_machine.maximum IS DISTINCT FROM spark.maximum) AS extremes_differents;
```

5.4 Lister les six moyennes qui ne tombent pas pareil. Fenêtre 3 :

```
-- Attendu : six lignes, toutes à 0,001 d'écart, celles du tableau ci-dessous.
WITH une_machine AS (SELECT etiquette, jour, points, moyenne FROM 'comparaison/duckdb/agregats.parquet'),
     spark AS (SELECT TagName AS etiquette, jour, moyenne FROM 'comparaison/spark/agregats/*.parquet')
SELECT une_machine.etiquette, une_machine.jour, une_machine.points,
       une_machine.moyenne AS sur_une_machine, spark.moyenne AS sur_spark
```


© 2026 iAlexMG. Tous droits réservés.

```
FROM une_machine JOIN spark USING (etiquette, jour)
WHERE une_machine.moyenne IS DISTINCT FROM spark.moyenne
ORDER BY une_machine.etiquette;
```

Étiquette	Jour	Points	Une machine	Spark
LHR.CNV.VIBRATION	2025-01-03	40	0,503	0,504
LMM.CNV.PUISSANCE	2025-02-24	40	1,687	1,686
PEI.CAB.PUISSANCE	2025-12-09	40	89,124	89,123
UTL.CHF.TEMP_ALIMENTATION	2025-07-16	48	27,347	27,348
UTL.CMP1.VIBRATION	2025-04-25	92	2,027	2,028
UTL.SEC.POINT_ROSEE	2024-10-13	40	16,148	16,149

À retenir — un contrôle d'égalité sur des flottants trouvera toujours un écart. Aucun des deux moteurs ne s'est trompé : une somme de flottants dépend de l'ordre des additions, et l'arrondi à trois décimales bascule quand le résultat exact tombe près d'un demi-millième. La bonne forme est une tolérance, `ABS(a - b) <= 0.001`, ou une comparaison sur les comptes et les extrêmes, qui sont exacts. Le Cours 11 en fera un contrat de données.

Point d'arrêt. Le même travail a tourné des deux côtés, et vous savez ce qu'il coûte. Qui fait ce labo en deux séances s'arrête ici. À la reprise, refaire « Avant de commencer » et rouvrir la fenêtre 3 comme en 2.3 : les étapes 6 à 8 ne supposent rien de plus que ce projet.

Étape 6 — Dédoublonner la télémétrie, le pire cas pour Spark

Théorie : `Theorie_07_DE.md`, annexe A.6.

6.1 Dédoublonner sur une machine, chronomètre en marche : un message par identifiant, le premier reçu. Fenêtre 1 :

```
duckdb -cmd ".timer on" -c "COPY (SELECT jour, message.message_id AS message_id, message.sernr AS sernr, recu_le FROM
'plateforme/lac/bronze/telemetrie/*/*.parquet' QUALIFY ROW_NUMBER() OVER (PARTITION BY message.message_id ORDER BY recu_le) = 1)
TO 'comparaison/duckdb/telemetrie.parquet' (FORMAT parquet);"
REM Attendu : Run Time (s): real 0.33 environ.
```

6.2 Contrôler ce qu'une machine a gardé. Fenêtre 3 :

```
-- Attendu : 201542 messages pour 201542 identifiants.
SELECT COUNT(*) AS messages, COUNT(DISTINCT message_id) AS identifiants
FROM 'comparaison/duckdb/telemetrie.parquet';
```

6.3 Écrire le même dédoublonnage en PySpark. Fenêtre 1 :

© 2026 iAlexMG. Tous droits réservés.

```
notepad 01_de\spark\telemetrie_dedoublonnee.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""telemetrie_dedoublonnee.py : un message par identifiant, en Spark (Cours 07)."""
import time

from pyspark.sql import SparkSession, functions as F
from pyspark.sql.window import Window

BRONZE = "plateforme/lac/bronze/telemetrie"
SORTIE = "comparaison/spark/telemetrie"

spark = SparkSession.builder.appName("boreal-telemetrie").getOrCreate()
spark.sparkContext.setLogLevel("WARN")

# La fenêtre : un groupe par identifiant de message, ordonné par la réception.
fenetre = Window.partitionBy("message.message_id").orderBy("recu_le")

dedoublonne = (spark.read.parquet(BRONZE)
               .withColumn("rang", F.row_number().over(fenetre))
               .filter(F.col("rang") == 1)
               .select("jour",
                       F.col("message.message_id").alias("message_id"),
                       F.col("message.sernr").alias("sernr"),
                       "recu_le"))

debut = time.perf_counter()
dedoublonne.write.mode("overwrite").parquet(SORTIE)
print(f"cours 07 | dédoublonnage : {time.perf_counter() - debut:.1f} s")

# Contrôle : autant de messages gardés que d'identifiants distincts.
# Attendu : 201 542 messages pour 201 542 identifiants, 933 jours, 7 644 véhicules.
c = (spark.read.parquet(SORTIE)
     .agg(F.count("*").alias("messages"),
          F.countDistinct("message_id").alias("identifiants"),
          F.countDistinct("jour").alias("jours"),
          F.countDistinct("sernr").alias("vehicules"))
     .collect()[0])
print(f"cours 07 | gardés : {c['messages']}, identifiants : {c['identifiants']}, "
      f"jours : {c['jours']}, véhicules : {c['vehicules']}")
```


© 2026 iAlexMG. Tous droits réservés.

```
spark.stop()
```

6.4 Lancer le dédoublonnage. Fenêtre 2 :

```
bash 01_de/spark/spark.sh 01_de/spark/telemetrie_dedoublonnee.py
# Attendu : dédoublonnage : une dizaine de secondes, puis gardés : 201542, identifiants : 201542,
# jours : 933, véhicules : 7644.
```

Étape 7 — Les trois pièges, mesurés

Théorie : `Theorie_07_DE.md`, annexe A.7.

7.1 Écrire le script qui provoque exprès ce qu'on passe sa carrière à éviter. Fenêtre 1 :

```
notepad 01_de\spark\pieges.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""pieges.py : les trois pièges du traitement distribué, mesurés (Cours 07)."""
import time

from pyspark.sql import SparkSession, functions as F

BRONZE = "comparaison/spark/historien"

spark = SparkSession.builder.appName("boreal-pieges").getOrCreate()
spark.sparkContext.setLogLevel("WARN")
bronze = spark.read.parquet(BRONZE)


def agreger(source):
    """Le même agrégat que celui du cours, réduit à son compte de points."""
    return (source.filter(F.col("Quality") == 192)
            .groupBy("TagName", F.to_date("Timestamp").alias("jour"))
            .agg(F.count("*").alias("points")))


def chrono(nom, action):
    debut = time.perf_counter()
    action()
    print(f"cours 07 | {nom} : {time.perf_counter() - debut:.1f} s")
```


© 2026 iAlexMG. Tous droits réservés.

```
# Piège 1 : un brassage de plus, pour rien.
chrono("avec un repartition de trop", lambda: agreger(bronze.repartition(200, "TagName"))
       .write.mode("overwrite").parquet("comparaison/spark/piege_repartition"))
chrono("le même, sans repartition", lambda: agreger(bronze)
       .write.mode("overwrite").parquet("comparaison/spark/piege_sans"))

# Piège 2 : la partition déséquilibrée.
tailles = (bronze.repartition(58, "TagName")
           .withColumn("partition", F.spark_partition_id())
           .groupBy("partition").count()
           .orderBy(F.desc("count")).collect())
print(f"cours 07 | partitions non vides : {len(tailles)} sur 58 demandées")
print(f"cours 07 | la plus grosse : {tailles[0]['count']} lignes, "
      f"la plus petite : {tailles[-1]['count']}")

bavardes = bronze.groupBy("TagName").count().orderBy(F.desc("count")).limit(3).collect()
discretes = bronze.groupBy("TagName").count().orderBy("count").limit(3).collect()
print("cours 07 | les plus bavardes :", [(r["TagName"], r["count"]) for r in bavardes])
print("cours 07 | les plus discrètes :", [(r["TagName"], r["count"]) for r in discretes])

# Piège 3 : l'action qui ramène tout au pilote.
debut = time.perf_counter()
une = bronze.filter(F.col("TagName") == "UTL.CMP3.MARCHE").collect()
print(f"cours 07 | collect d'une étiquette : {len(une)} lignes en "
      f"{time.perf_counter() - debut:.1f} s")

debut = time.perf_counter()
tout = bronze.collect()
print(f"cours 07 | collect de tout : {len(tout)} lignes en {time.perf_counter() - debut:.1f} s")

spark.stop()
```

7.2 Lancer les trois pièges. Fenêtre 2 :

```
bash 01_de/spark/spark.sh 01_de/spark/pieges.py
# Attendu : les huit lignes cours 07 ci-dessous, aux durées près.

cours 07 | avec un repartition de trop : 17.4 s
cours 07 | le même, sans repartition : 8.5 s
cours 07 | partitions non vides : 37 sur 58 demandées
cours 07 | la plus grosse : 247879 lignes, la plus petite : 1833
```


© 2026 iAlexMG. Tous droits réservés.

```
cours 07 | les plus bavardes : [('ESS.BANC.TEMP_REFROIDISSEMENT', 242202), ('UTL.CHF.PUISSANCE', 188499), ('PEI.CAB.HUMIDITE',
71544)]
cours 07 | les plus discrètes : [('UTL.CMP3.MARCHE', 1833), ('LMM.CNV.MARCHE', 2378), ('LMN.ROB.MARCHE', 2501)]
cours 07 | collect d'une étiquette : 1833 lignes en 7.4 s
cours 07 | collect de tout : 2407796 lignes en 19.8 s
```

Le piège	Ce que le script fait	Chez l'auteur
le brassage de trop	le même agrégat, précédé ou non d'un `repartition` sur l'étiquette	17,4 s contre 8,5 s
la partition déséquilibrée	le bronze redistribué sur l'étiquette, en 58 partitions	37 non vides, de 1 833 à 247 879 lignes
l'action qui ramène tout	`collect` d'une étiquette, puis de tout le bronze	1 833 lignes en 7,4 s, 2 407 796 en 19,8 s

Étape 8 — Conclure, remettre le projet d'aplomb, versionner

Théorie : `Theorie_07_DE.md`, annexe A.8.

8.1 Reporter vos durées à côté de celles de l'auteur :

Le travail	Une machine	Spark	Rapport
bronze de l'historien, 2 407 796 lignes	2,7 s	46 à 77 s	17 à 29 fois
agrégats quotidiens, 47 618 lignes	0,19 s	10,6 s	55 fois
dédoublonnage de la télémétrie, 201 542 messages	0,33 s	11,0 s	33 fois
ce que le lancement coûte en plus	rien	une vingtaine de secondes de démarrage du conteneur	

Sur ce volume, le moteur d'une machine gagne de deux ordres de grandeur, et il gagne aussi en simplicité d'exploitation. Spark reste dans le dépôt parce qu'il faut savoir l'écrire, pas parce que Boréal en a besoin.

8.2 Fermer l'interface web : `.exit` à l'invite `D` de la fenêtre 3, puis l'onglet du navigateur. Attendu : l'invite de l'Anaconda Prompt revient.

8.3 Régénérer les sources dans l'état du Cours 06, sans quoi le Cours 08 ne retrouvera pas ses chiffres. Fenêtre 1 :

© 2026 iAlexMG. Tous droits réservés.

```
python ..\_univers\boreal\generer_boreal.py --projet . --jusqua 2026-03
REM Attendu, en une minute et demie environ : le bilan, dont historien 182 jours, puis
REM 1 989 fichiers, 177,5 Mo, empreinte globale 2081493759980946..., et Conforme à la référence
REM des cours.
```

8.4 Refaire le bronze, qui dérive des sources. Fenêtre 1 :

```
python -m pipeline bronze
REM Attendu : historien 550 087 lignes et télémétrie 204 485 lignes, source et bronze d'accord.
```

Le bronze de l'historien garde 639 dossiers `jour=` vides, du 2024-01-01 au 2025-09-30 : `OVERWRITE` efface les fichiers, pas les dossiers. Sans effet sur la suite, qui ne lit que les `*.parquet`.

8.5 Effacer le dossier de comparaison, qui ne sert plus. Fenêtre 1 :

```
rmdir /s /q comparaison
REM Attendu : rien.
```

8.6 Écrire le fichier qui empêche Git de remettre les retours chariot de Windows dans le lanceur. Fenêtre 1 :

```
notepad 01_de\.gitattributes
```

Accepter la création. Coller, enregistrer, fermer :

```
# Les scripts shell gardent des fins de ligne à la mode d'Unix, même sur un poste Windows :
# bash refuse un retour chariot en fin de ligne.
*.sh text eol=lf
```

8.7 Versionner le code de Spark. Fenêtre 1 :

```
git -C 01_de add spark .gitattributes
REM Attendu : rien.
git -C 01_de status
REM Attendu : Changes to be committed, six fichiers en new file : .gitattributes et les cinq de
REM spark. Rien de comparaison, qui vivait hors du dépôt.
git -C 01_de commit -m "Cours 07 : les memes traitements en PySpark, et les trois pieges mesures"
REM Attendu : 6 files changed.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
```

Pour tout fermer : `exit` dans la fenêtre 2, qui redevient une Anaconda Prompt. Aucun conteneur ne reste : le lanceur les efface à la fin de chaque lancement.

© 2026 iAlexMG. Tous droits réservés.

Dépannage

Symptôme	Correctif
`No module named pipeline`	la fenêtre 1 n'a pas son `set PYTHONPATH=01_de` (fenêtre rouverte) : le retaper, puis relancer
`python: can't open file '…\_univers\boreal\generer_boreal.py'`	la fenêtre 1 n'est pas dans `Projet_Boreal_intermediaire` : la rouvrir comme au début
`docker: command not found` dans Ubuntu	Docker Engine n'est pas installé dans cette distribution : étape 4 du `Data_Domain/00_Installation/Guide_installation.md`
`docker: permission denied while trying to connect`	le compte Ubuntu n'est pas dans le groupe `docker` : `sudo usermod -aG docker $USER`, puis rouvrir Ubuntu
`docker ps` montre les services d'Airflow	la pile du Cours 06 tourne encore et prend des cœurs à Spark : `docker compose -f plateforme/docker-compose.yml stop`
Le Bloc-notes ouvre un fichier qui a déjà du contenu en 3.2, 3.3, 5.1, 6.3, 7.1 ou 8.6	un reste d'un premier essai : tout sélectionner, coller le fichier à sa place, enregistrer
`bash: 01_de/spark/spark.sh: No such file or directory`	le Bloc-notes a enregistré `spark.sh.txt` (`dir 01_de\spark` le montre) : `ren 01_de\spark\spark.sh.txt spark.sh`
`dir` répond `Fichier introuvable` sur un dossier `jour=…`	le chemin n'est pas entre guillemets : l'Anaconda Prompt le coupe au signe `=`
`set: -: invalid option` ou `$'\r': command not found`	le lanceur a gardé les fins de ligne de Windows : la commande de 3.5, puis relancer
`java.io.FileNotFoundException: File sources/usine/historien/*.csv does not exist`	rien : Spark sonde le chemin littéral avant d'étendre le joker ; le compte final le prouve
`chmod: changing permissions of '/projet/comparaison/spark/historien': Operation not permitted`	le conteneur tourne sous un autre utilisateur que le propriétaire du montage : garder `--user root` dans `spark.sh`
`java.io.IOException: Invalid UID, could not determine effective user`	`--user` désigne un identifiant qui n'existe pas dans l'image : `--user root`, le seul compte sûr de cette image
`Py4JJavaError` sur `Path does not exist: file:/projet/...`	la fenêtre 2 n'est pas dans `Projet_Boreal_intermediaire` : la fermer, puis la rouvrir comme au début

© 2026 iAlexMG. Tous droits réservés.

Symptôme	Correctif
`AnalysisException: [PATH_ALREADY_EXISTS]`	une écriture sans `mode("overwrite")` : ajouter le mode, comme dans les scripts du labo
les agrégats de Spark ne comptent pas comme ceux d'une machine	les deux bronzes ne viennent pas de la même génération : refaire l'étape 1 en entier, puis 3.6
`python -m pipeline bronze` rend 550 087 lignes d'historien en 1.2	les sources n'ont pas été régénérées avec `--historien complet` : refaire 1.1
`rmdir` répond `Le processus ne peut pas accéder au fichier`	l'interface web tient encore un fichier : 8.2, puis relancer

Annexe — Sur Databricks

Pour qui veut voir le même code sur la plateforme que l'industrie nomme le plus souvent quand elle dit « Spark ». Les faits ont été vérifiés dans la documentation de Databricks en septembre 2026.

Databricks Free Edition suffit pour cette annexe, et ses limites décident de ce qu'on peut montrer :

La limite	Ce qu'elle change
calcul serverless seulement, aucune configuration de calcul personnalisée	le dimensionnement d'une grappe, le nombre d'exécuteurs et leur mémoire ne se règlent pas, donc ne se montrent pas
R et Scala non pris en charge	l'annexe est en PySpark, comme le labo
un seul entrepôt SQL, en taille 2X-Small	les mesures de durée n'ont rien à voir avec celles du labo, et ne se comparent pas
cinq tâches de travail simultanées par compte	les essais se font un à la fois
quota dépassé, le calcul s'arrête pour le reste de la journée	les données et les réglages restent intacts ; le travail reprend le lendemain
aucun usage commercial	pour apprendre, pas pour un mandat

Le code du labo tourne tel quel, à trois détails près. La session existe déjà : dans un notebook Databricks, la variable `spark` est fournie, et les lignes `SparkSession.builder` ne servent plus. Les chemins changent : les fichiers vivent dans un volume Unity Catalog, `/Volumes/<catalogue>/<schéma>/<volume>/…`, et non dans un dossier du poste. Et l'écriture se fait en Delta plutôt qu'en Parquet nu, parce que c'est le format par défaut de la plateforme.

```
# Dans un notebook Databricks : « spark » existe déjà.
from pyspark.sql import functions as F, types as T
```


© 2026 iAlexMG. Tous droits réservés.

```
SCHEMA = T.StructType([
    T.StructField("TagName", T.StringType()),
    T.StructField("Timestamp", T.TimestampType()),
    T.StructField("Value", T.DoubleType()),
    T.StructField("Quality", T.IntegerType()),
])

historien = (spark.read
             .option("header", True)
             .option("timestampFormat", "yyyy-MM-dd HH:mm:ss")
             .schema(SCHEMA)
             .csv("/Volumes/<catalogue>/bronze/livraisons/usine/historien/")
             .withColumn("jour", F.to_date(F.regexp_extract(
                 F.input_file_name(), r"historien_(\d{4}-\d{2}-\d{2})", 1))))

(historien.write.mode("overwrite").partitionBy("jour")
 .saveAsTable("bronze.usine_historien"))
```

Ce que la plateforme apporte par-dessus le moteur, et que le labo ne peut pas montrer en local : le catalogue et ses permissions, l'historique des versions d'une table Delta, le suivi des traitements, et le fait que la donnée vive déjà à côté du calcul. Ce que le labo montre et que la plateforme cache : ce que coûte un brassage, et combien de partitions un travail crée. Les deux savoirs se complètent, et c'est le second qui fait diagnostiquer une lenteur.

Le format Delta, le voyage dans le temps et le catalogue reviennent au Cours 10, qui les traite pour de bon.

© 2026 iAlexMG. Tous droits réservés.